Arduino for Embedded Systems - Beginner Level

Lesson 2: Basic Arduino Programming Concepts

Basic Arduino Programming Concepts - Study Notes

Key Concepts

• **Variables & Data Types** – storing values (int, float, char, bool, etc.) and choosing the right type for memory efficiency.

• **Control Structures** – `if…else`, `for`, `while`, and `switch` statements to direct program flow.

• **Serial Communication** – using `Serial.begin()`, `Serial.print()`, `Serial.println()`, and `Serial.available()` to exchange data with a PC.

• **Setup & Loop Functions** – the required skeleton of every Arduino sketch (`void setup()` runs once, `void loop()` repeats).

• **Basic I/O** – reading digital/analog pins (`pinMode()`, `digitalRead()`, `analogRead()`) and writing outputs (`digitalWrite()`, `analogWrite()`).

Summary

Arduino programming is built on a simple C/C++ like language that lets you interact with hardware through variables, control flow, and serial communication. A sketch always begins with two mandatory functions: `setup()`, where you initialize pins, serial ports, and variables, and `loop()`, where the main program repeats indefinitely. Variables hold data such as sensor readings or counters; picking the appropriate data type (e.g., `uint8_t` for 0 255 values) conserves the limited RAM on most Arduino boards. Control structures let you make decisions (`if`/`else`), repeat actions (`for`/`while`), or handle multiple cases (`switch`). Finally, the Serial library enables debugging and PC Arduino interaction by sending text or binary data over USB, which appears in the Arduino IDE's Serial Monitor. Mastering these fundamentals provides the foundation for building more complex projects involving sensors, actuators, and communication protocols.

Important Points

• **Variable declaration**: `type name = value;` (e.g., `int ledPin = 9;`).

• **Data type ranges** (on AVR based Arduinos):

- `bool`: true/false
- `char`: -128 to 127
- `unsigned char` / `byte`: 0 to 255
- `int`: -32,768 to 32,767
- `unsigned int`: 0 to 65,535
- `long`: -2,147,483,648 to 2,147,483,647
- `float`: ~6 7 decimal digits of precision

• **Setup()** runs once after power up or reset; use it for `pinMode()`, `Serial.begin()`, and initial

variable assignments.

• **Loop()** runs continuously; avoid long blocking delays if you need responsive behavior (consider `millis()` for non blocking timing).

• **Serial communication**:

- `Serial.begin(9600);` sets baud rate (must match Serial Monitor).
- `Serial.print()` sends data without newline; `Serial.println()` adds `\r\n`.
- `Serial.available()` returns number of bytes ready to read; `Serial.read()` fetches one byte.

• **Control structures**:

- `if (condition) { … } else { … }`
- `for (init; test; increment) { … }`
- `while (condition) { … }`
- `switch (variable) { case value: … break; default: … }`

• **Pin modes**: `INPUT`, `OUTPUT`, `INPUT_PULLUP` (enables internal pull up resistor).

• **Analog vs. Digital**: `analogRead()` returns 0 1023 (10 bit ADC); `analogWrite()` uses PWM (0 255) on PWM capable pins.

Examples

Example 1: Blink LED with Variable Delay

```cpp
// Pin where the LED is connected
const int ledPin = 13;
// Delay time in milliseconds (can be changed while running)
unsigned int blinkDelay = 500;
void setup() {
  pinMode(ledPin, OUTPUT);   // set LED pin as output
  Serial.begin(9600);        // start serial for debugging
  Serial.println("Blink sketch started");
}
void loop() {
  digitalWrite(ledPin, HIGH);   // turn LED on
  Serial.println("LED ON");
  delay(blinkDelay);            // wait
  digitalWrite(ledPin, LOW);    // turn LED off
  Serial.println("LED OFF");
  delay(blinkDelay);            // wait
  // Example of changing delay via Serial input
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'f') {          // 'f' = faster
      blinkDelay = max(50, blinkDelay - 100);
    } else if (cmd == 's') {   // 's' = slower
```

```
      blinkDelay = min(2000, blinkDelay + 100);
    }
    Serial.print("New delay: ");
    Serial.println(blinkDelay);
  }
}
```

Explanation

- `ledPin` is a constant (`const`) because it never changes.
- `blinkDelay` is an `unsigned int` to store a non negative delay value.
- `setup()` configures the pin as an output and starts serial communication at 9600 baud.
- `loop()` turns the LED on/off, prints status to the Serial Monitor, and waits using `delay()`.
- When the user sends `'f'` or `'s'` via the Serial Monitor, the delay is adjusted, demonstrating serial input and variable modification.

Example 2: Reading a Potentiometer and Controlling LED Brightness

```cpp
const int potPin = A0;   // analog pin connected to potentiometer
const int ledPin = 9;    // PWM pin for LED brightness
void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  int sensorValue = analogRead(potPin);   // 0  1023
  int brightness = map(sensorValue, 0, 1023, 0, 255); // scale to PWM range
  analogWrite(ledPin, brightness);        // set LED brightness
  // Optional: print values for debugging
  Serial.print("Sensor: ");
  Serial.print(sensorValue);
  Serial.print("\t Brightness: ");
  Serial.println(brightness);
  delay(20); // small delay to stabilize reading
}
```

Explanation

- `analogRead()` reads the voltage on `A0` (0 5 /V) and returns 0 1023.
- `map()` re scales that range to 0 255, suitable for `analogWrite()` (PWM).
- The LED brightness changes smoothly as the potentiometer is turned.
- Serial output shows both raw sensor value and computed brightness, useful for troubleshooting.

Quick Review

1. **What is the purpose of the `setup()` function in an Arduino sketch?**
   - It runs once at start up to initialize pins, serial communication, and variables.
2. **Which data type would you use to store a value that only needs to hold 0 255, and why?**
   - `byte` or `unsigned char` – it uses only 1 /byte of RAM and matches the required range.
3. **How do you make an Arduino wait for 1 second without blocking other code?**
   - Use `millis()` to track elapsed time and perform actions when the interval has passed, rather than `delay(1000)`.
4. **What does `Serial.available()` return, and when would you use it?**
   - It returns the number of bytes received in the serial buffer; use it to check if incoming data is ready to be read before calling `Serial.read()`.
5. **Explain the difference between `digitalWrite()` and `analogWrite()`.**
   - `digitalWrite()` sets a pin to HIGH (5 /V) or LOW (0 /V). `analogWrite()` outputs a PWM signal with a duty cycle from 0 255, simulating analog voltage levels on PWM capable pins.

Further Reading

- **Arduino Language Reference** – official docs for `pinMode()`, `digitalRead/Write()`, `analogRead/Write()`, `Serial`, and control structures.
- **Using `millis()` for Timing** – tutorial on non blocking delays and state machines.